252-0027

Einführung in die Programmierung

Übungen

Woche 5: Arrays, Methoden, Debugger

Joran Bühler
Departement Informatik
ETH Zürich

Inhalt

- Organisatorisches
- Arrays
- Methoden
- Value vs. Reference
- Rekursion
- Kahoot
- Debugger
- Vorbesprechung

Das Wichtigste zur ersten Bonusaufgabe

- **Java 25**
- **Es muss kompilieren! Letzer Push vor Abgabetermin zählt!**
  - Auch auf die Outputs achten: Gross/Kleinschreibung zählt auch.
  - Random print() Anweisungen löschen.
- **Keine Methoden/Klassen-Namen verändern.**
- **Keine static-Attribute verwenden!**
- **Versucht die Aufgabe selbst zu lösen.**
- https://lec.inf.ethz.ch/infk/eprog/2026/exercises/additionals/bonusaufgaben_regeln.pdf

Timed Bonus

- **Jede Woche eine Bonusaufgabe**
- **Ausser:**
  - 27.10. und 1.12.
  - -> Dienstag Abend der Folgewoche (**3.11**. bzw **8.12.**)
  - **17:00 – 19:00**
- **Mehr Infos folgen.**

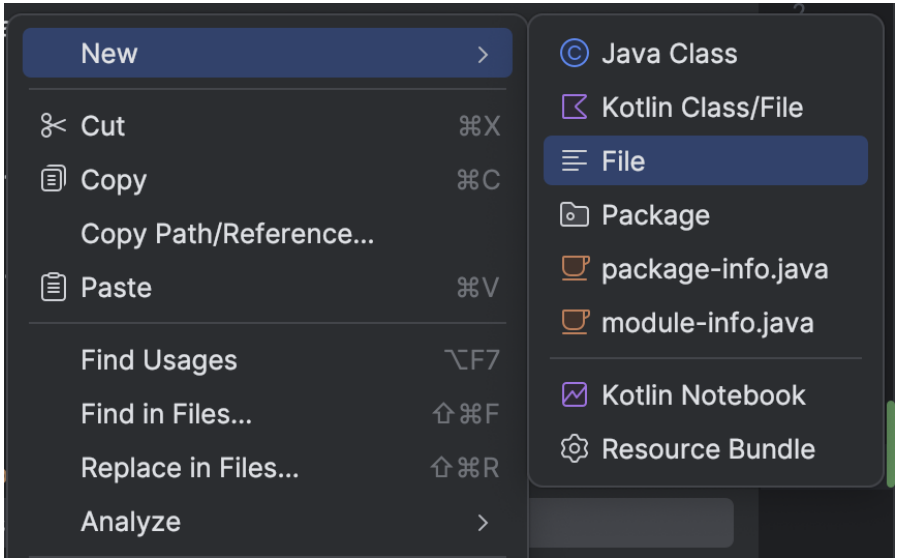
Feedback

Feedback

- Ihr sagt den TAs ab u05 wo ihr Feedback haben möchtet.
- Dazu erstellt ihr eine requestfeedback.txt Datei (in den uXX Ordner – nicht src / test / resources).
- In die Datei schreibt ihr die Aufgaben, für welche ihr Feedback haben wollt. (**Bonus wird separat gegraded, aber ich gebe euch auch gerne Feedback dazu – wenn Ihr wollt)**

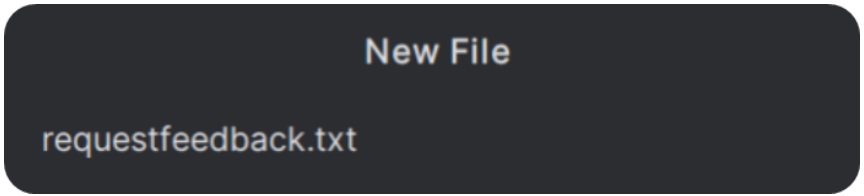
Feedback Datei erstellen

1. **Projekt Ordner uXX rechts-klicken.**
2. **New -> File.**



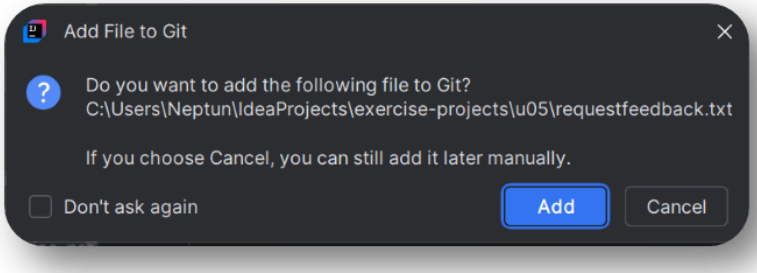
Feedback Datei erstellen

1. **Projekt Ordner uXX rechts-klicken.**
2. **New -> File.**
3. **Benenne die Datei als requestfeedback.txt im erscheinenden Fenster.**



Feedback Datei erstellen

1. **Projekt Ordner uXX rechts-klicken.**
2. **New -> File.**
3. **Benenne die Datei als requestfeedback.txt im erscheinenden Fenster.**
4. **Wähle *Add* im auftauchenden Fenster «Add File to Git»**



1D Arrays

Eindimensionale Arrays

- Arrays belegen eine feste Größe **n** im Speicher, haben daher auch eine feste Länge
- für jeden Eintrag kann für den deklarierten Datentyp ein Wert gespeichert werden
- auf ein spezifisches Element **i** können wir mit **arr[i]** zugreifen
- Indizierung startet bei **0**, geht also bis **n-1**

```java
public class Main {
    public static void main(String[] args){
        int[] arr = {3,5,6,1,2,8};
        for(int i = 0; i < arr.length; i++){
            arr[i] *= 2;
        }
    }
}
```

2D Arrays

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

[[1,2,3],[4,5,6],[7,8,9]]

Zweidimensionale Arrays

- eine Matrix könnte als Zweidimensionales Array dargestellt werden
- **int[][] arr = new int[n][n];**

  definiert ein Array, welches **n** integer-Arrays der Länge **n** speichert **(nxn-Matrix)**
- eine gesamte Zeile erhalten wir also mit **arr[i]** für **0≤i<n**
- einen Eintrag erhalten wir mit **arr[i][j]** für **0≤i,j<n**
- **Vorsicht:** Die Längen der Arrays könnten unterschiedlich sein!

```java
public class Main {
    public static void main(String[] args) {
        int[][] arr = {{1, 2, 3},{4, 5, 6},{7, 8, 9}};
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                arr[i][j] *= 2;
            }
        }
    }
}
```

2D Arrays

array = {arrOne, arrTwo, arrThree}

i	array[i]
0	
1	
2	

i	0	1	2	3	4	5
arrOne[i]	4	501	-1	200	42	0

i	0	1	2
arrTwo[i]	11	21	-170

i	0	1	2	3
arrThree[i]	100	1	-321	3

```
int[][] arTwoD = {
        {1,2,3},
        {4,5,6},
        {7,8,9}
};
```

arTwoD[i] =

i	
0	{1,2,3}
1	{4,5,6}
2	{7,8,9}

arTwoD[i][j] =

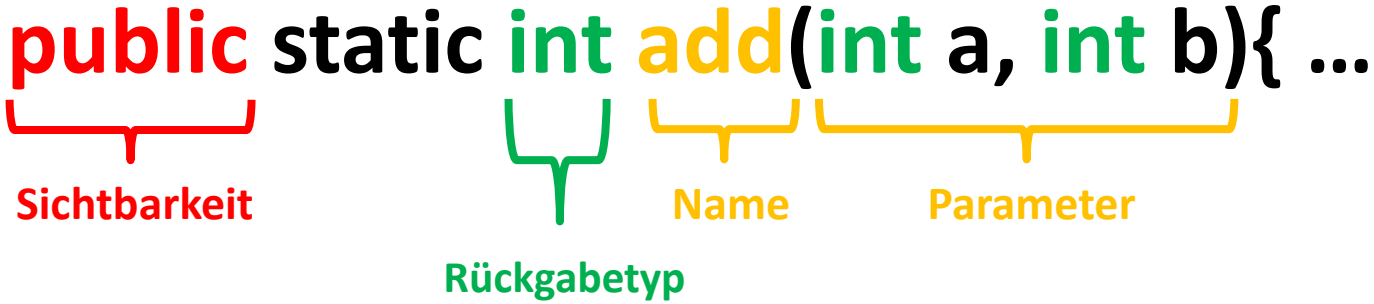
i\j	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

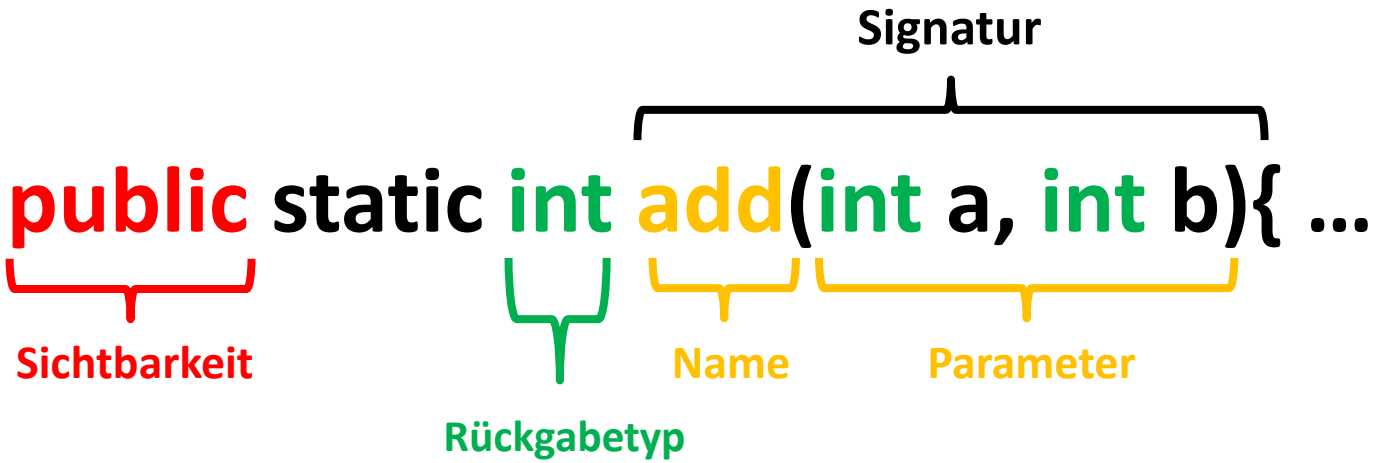
Nützliche Array Funktionen:

- **Arrays.toString(array) –** returns a string with the elements of a 1D array like [1, 2, 3].
- **Arrays.deepToString(array) –** returns a string for multidimensional arrays, printing nested arrays properly.
- **Arrays.equals(a, b) –** checks if two 1D arrays are equal element by element.
- **Arrays.deepEquals(a, b) –** checks equality for nested (multi-dimensional) arrays recursively.
- **Arrays.sort(array) –** sorts the array in ascending order

Methoden

```
public static int add(int a, int b){ …
```


public static int add(int a, int b){ …
Sichtbarkeit
Rückgabetyp
Name
Parameter


Signatur
public static int add(int a, int b){ …
Sichtbarkeit
Rückgabetyp
Name
Parameter

Method-Overloading

- Eine Methode kann überladen werden, in dem die Methodenparameter verändert werden
- trotz gleichem Namen hat sie dann eine andere Signatur
- **add** kann durch Überladung mit unterschiedlichen Parametern aufgerufen werden, hat nun unterschiedliche Rückgabetypen

```java
public class Main {
    // Erste Version: Addiert zwei int-Werte
    public static int add(int a, int b) {
        return a + b;
    }
    // Überladene Version: Addiert drei int-Werte
    public static int add(int a, int b, int c) {
        return a + b + c;
    }
    // Überladene Version: Addiert zwei double-Werte
    public static double add(double a, double b) {
        return a + b;
    }
}
```

Value vs Reference

Objekte 101 – (Deep Dive in einer späteren Übung)

- Klassen sind „Baupläne“, die definieren, wie Daten gespeichert werden
- Objekte sind konkrete Realisierungen der Klasse
- Die Variable des Objektes speichert einen Verweis, wo die Daten im Speicher liegen (Referenz)

```java
public class Main {
    public static void main(String[] args){
        Coordinate coord = new Coordinate(2,3);
        System.out.println(coord);
    }
}
public class Coordinate {
    int x;
    int y;
    public Coordinate(int xcoord, int ycoord){
        x = xcoord;
        y = ycoord;
    }
}
```

Primitive Datentypen

- Variablen speichern einen Wert, z. B. *int*, *double* oder *boolean*
- Haben einen fest definierten Wertebereich
- Standardwerte sind festgelegt
- Können nicht *null* sein

Referenztypen

- Variablen speichern eine Referenz auf ein Objekt (z.B. String oder Array) oder *null*
- Objekte können unterschiedlich viel Speicher benötigen
- Standardwert ist *null* (verweist auf kein Objekt)

Weitergabe von Referenzen

- ***numbers*** speichert die Referenz, wo die tatsächlichen Werte liegen
- gib Referenz an ***modifyArray*** weiter
- ***modifyArray*** sucht Werte im Speicher, verdoppelt sie, terminiert
- ***main*** sucht mit derselben im Speicher und findet verdoppelte Werte **ohne**, dass wir sie zurückgeben mussten

```
Vor der Änderung:
[1, 2, 3, 4, 5]
Nach der Änderung:
[2, 4, 6, 8, 10]
```

```java
public class ReferenceArrayDemo {
    public static void main(String[] args) {
        // Erstelle ein Array
        int[] numbers = {1, 2, 3, 4, 5};
        System.out.println("Vor der Änderung:");
        System.out.println(Arrays.toString(numbers));
        modifyArray(numbers); // Übergabe des Arrays
        System.out.println("Nach der Änderung:");
        System.out.println(Arrays.toString(numbers));
    }
    public static void modifyArray(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] *= 2; // Verdopple jeden Wert im Array
        }
    }
```

Ausnahmen - Unveränderlichkeit von Strings

- Strings sind Objekte
- Aufgrund der Unveränderlichkeit scheint es, als wären sie Pass-by-Value
- Das liegt an den Semantics von =

```java
public class StringDemo {
    public static void main(String[] args) {
        String a = "Hello";
        System.out.println(a); // Hello
        modifyString(a);
        System.out.println(a); // Hello
    }
    public static void modifyString(String a) {
        a = a+" World!";
        System.out.println(a); // Hello World!
    }
}
```

Rekursion

Endliche Rekursion in Java

- Mit Abbruch: einen Weg durch Methode ohne rekursiven Aufruf
  - **Basisfall** («**base case**»): Argument, das zu Abbruch führt
  - **Rekursiver Fall**: Argument, das zu rekursivem Aufruf führt

```
public static void f() {
    if (...) {
        ...
    }

    else {
        f();
    }
}
```

Basisfall

Rekursiver Fall

Rekursion - Beispiel

1. Implementieren Sie die Methode `Calculations.checksum(int x)`, das heisst die Methode `checksum` in der Klasse `Calculations`. Die Methode nimmt einen Integer `x` als Argument, welcher einen nicht-negativen Wert hat. Die Methode soll die Quersumme von `x` zurückgeben. Sie sollen für diese Aufgabe **keine** Schleife verwenden.

   Beispiele

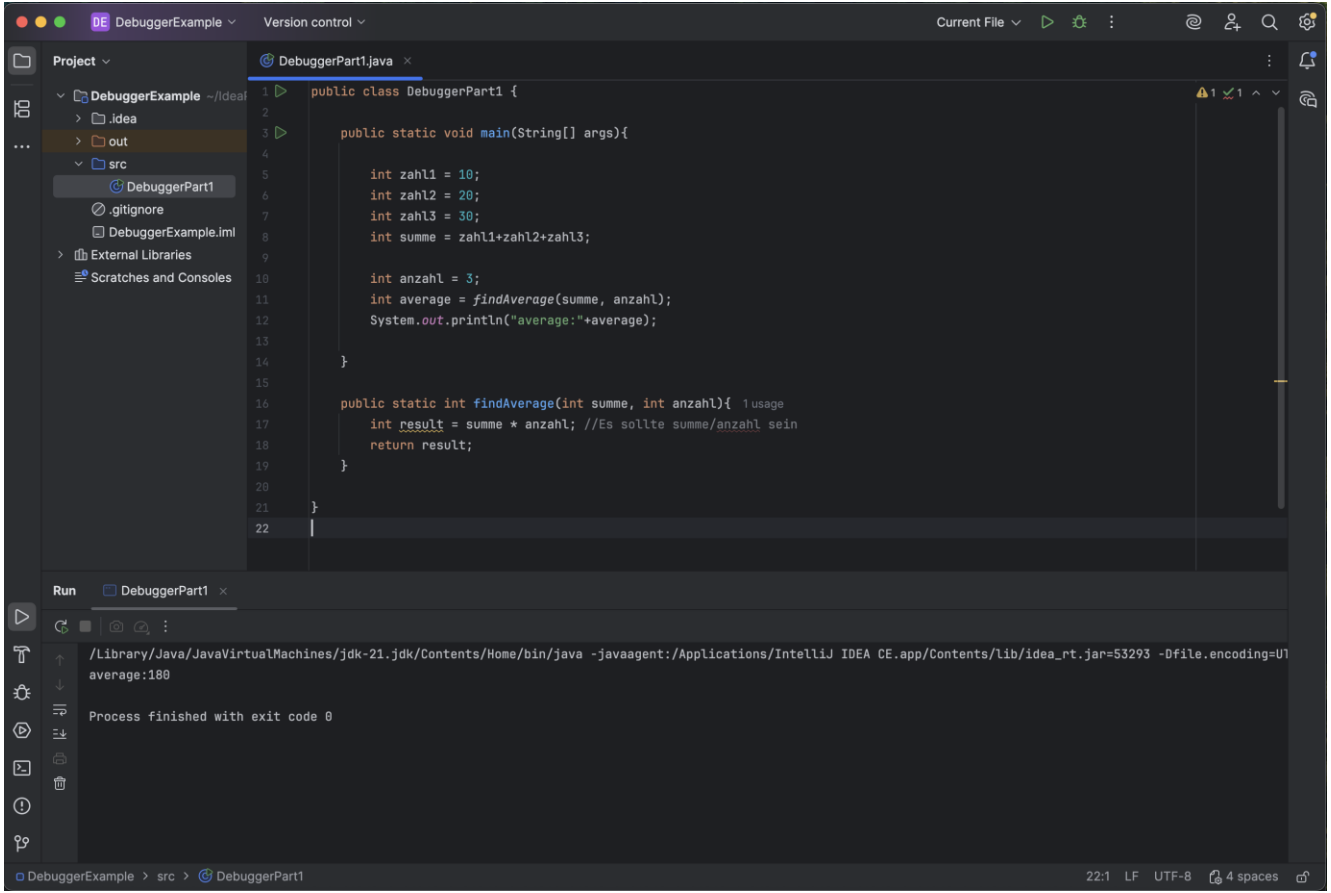
   - `checksum(258)` gibt 15 zurück.
   - `checksum(49)` gibt 13 zurück.
   - `checksum(12)` gibt 3 zurück.

   Hinweis: Für einen Integer a ist `a % 10` die letzte Ziffer und `a / 10` entfernt die letzte Ziffer. Zum Beispiel `258 % 10` ist 8 und `258 / 10` ist 25.

Java Debugger

Was ist der Debugger und was tut er?

- Ein Tool (in Java), das beim Debuggen hilft.
- Mit dem Java-Debugger kann man Schritt für Schritt durch das Programm gehen und genau beobachten, wie es ausgeführt wird.
- Die Änderungen der Variablen und ihrer Werte in jeder Zeile werden in einer Tabelle dargestellt.



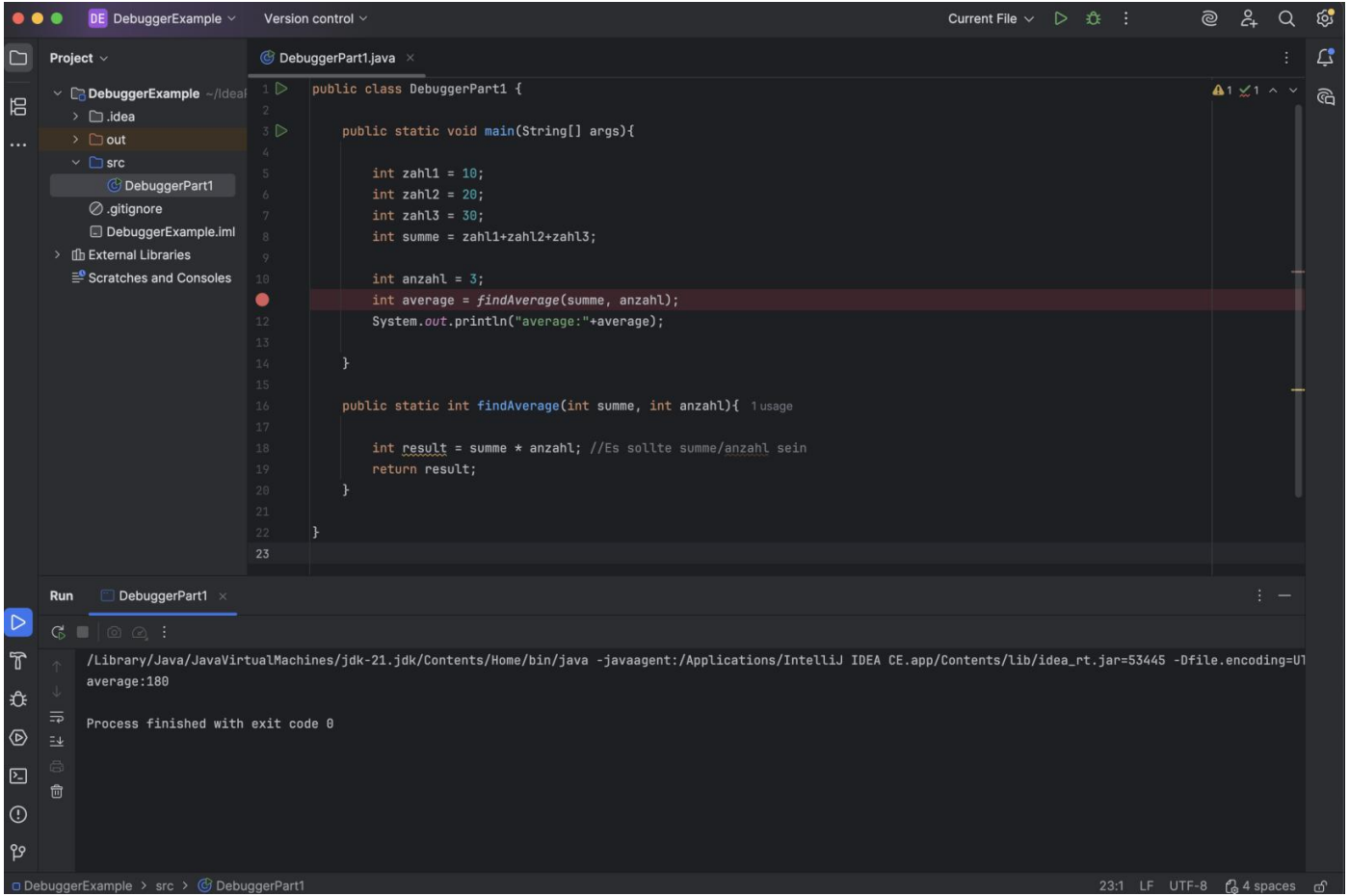
Das Programm macht nicht, was du erwartest, und du kannst den Fehler nicht finden?

Benutze den Debugger!

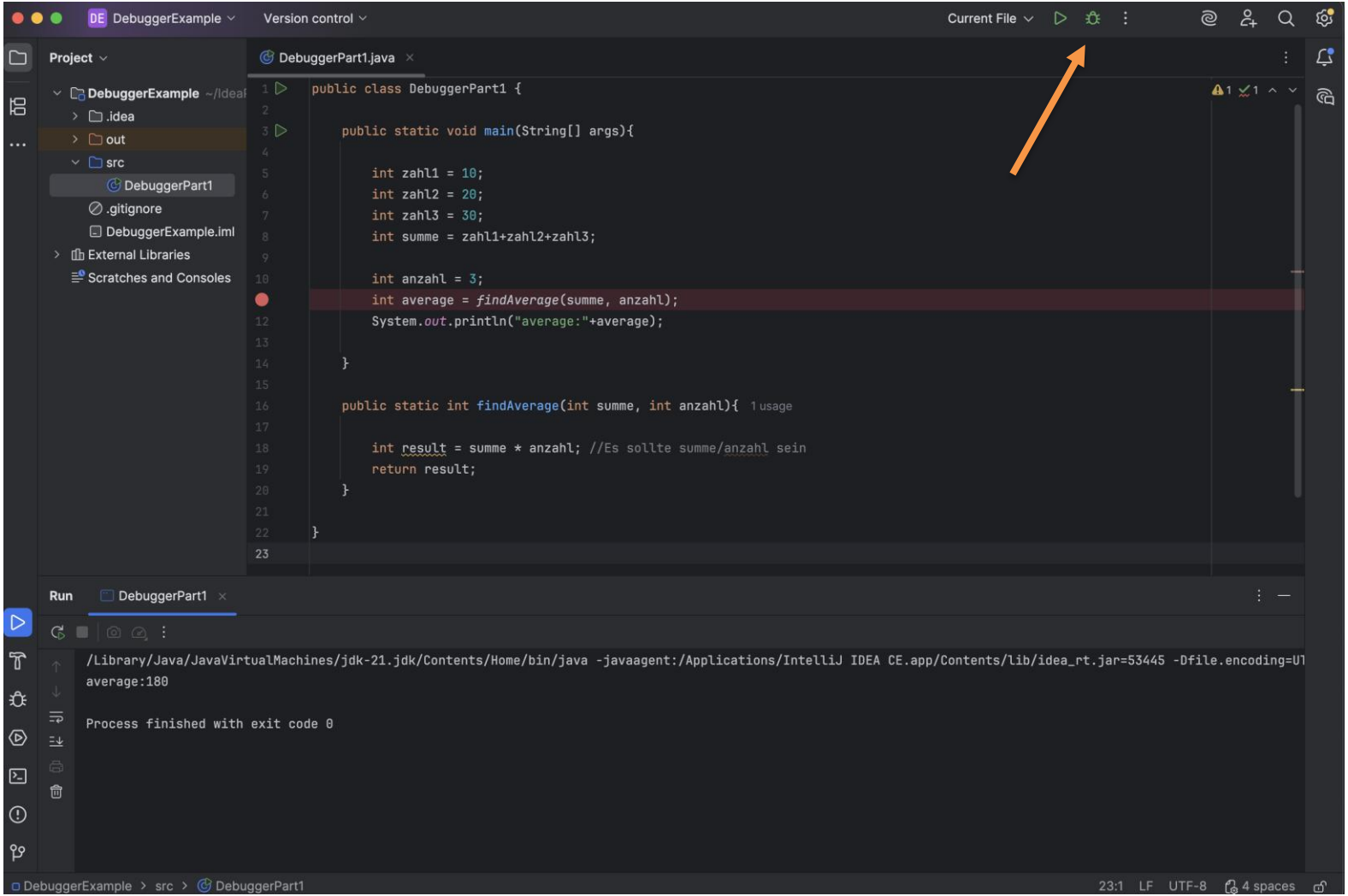
Breakpoints

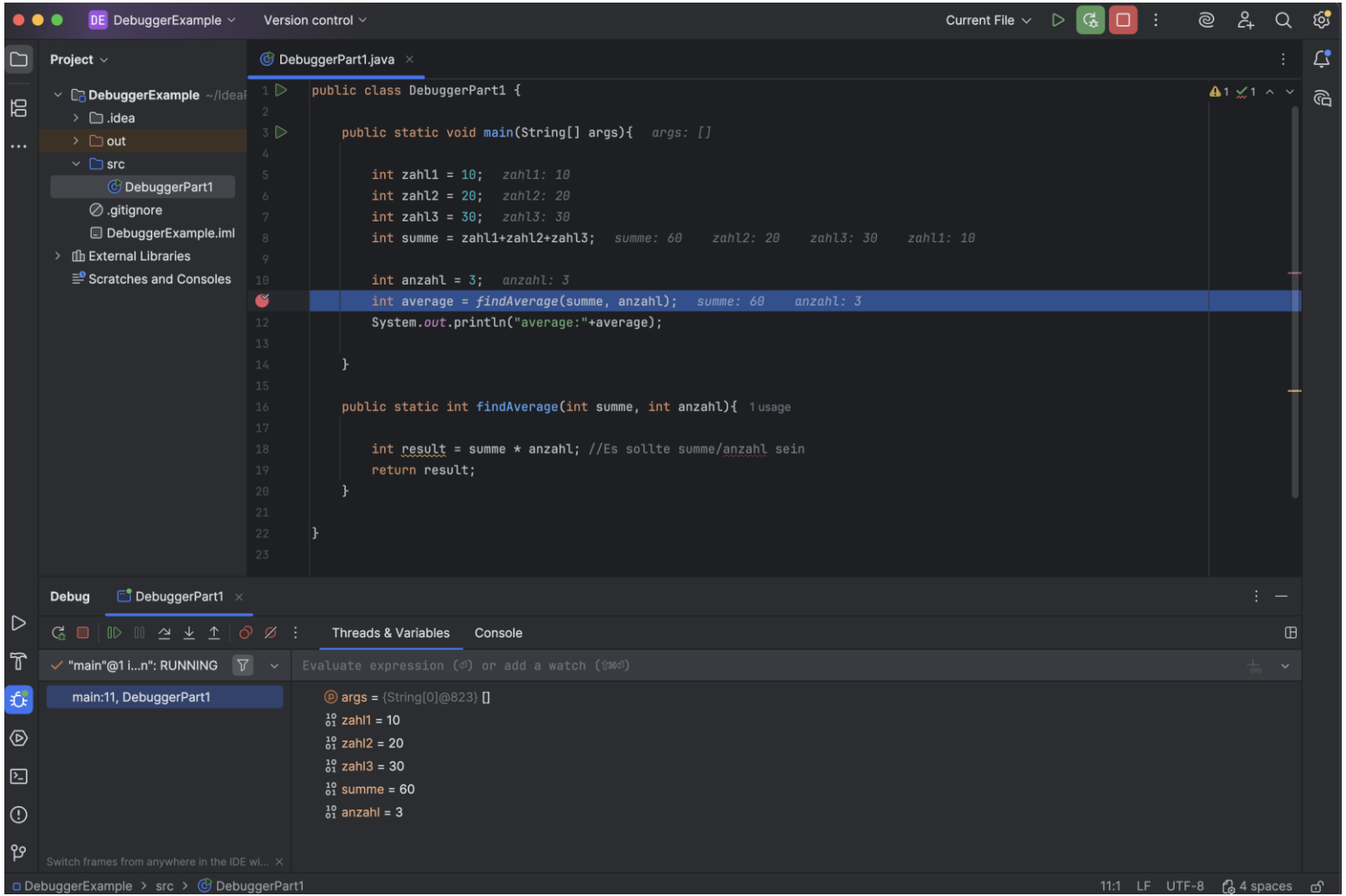
- Bis zum Breakpoint wird alles automatisch ausgeführt, und sobald die Zeile mit dem Breakpoint erreicht wird, stoppt die automatische Ausführung.
- Anschliessend kann der Benutzer Zeile für Zeile das Programm selbst ausführen.
- **Breakpoint auswählen:** An einer Stelle, an der man weiss, dass alles bis dahin wie erwartet funktioniert.
- **Breakpoint setzen:** Links von der Spalte mit den Zeilennummern klicken, um den Breakpoint zu setzen



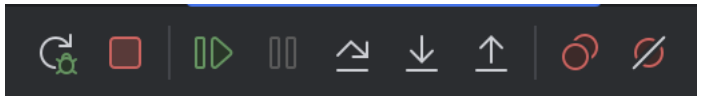


- Da wir sicher sind, dass **summe** und **anzahl** stimmen, wollen wir nun die Methode **findAverage** testen.
- Dazu setzen wir einen Breakpoint in der Zeile, in der diese Methode aufgerufen wird.


DebuggerExample
Version control
Current File
Project
DebuggerExample ~/IdeaP
.idea
out
src
DebuggerPart1
.gitignore
DebuggerExample.iml
External Libraries
Scratches and Consoles
DebuggerPart1.java
public class DebuggerPart1 {
public static void main(String[] args){
int zahl1 = 10;
int zahl2 = 20;
int zahl3 = 30;
int summe = zahl1+zahl2+zahl3;
int anzahl = 3;
int average = findAverage(summe, anzahl);
System.out.println("average:"+average);
}
public static int findAverage(int summe, int anzahl){ 1 usage
int result = summe * anzahl; //Es sollte summe/anzahl sein
return result;
}
}
Run
DebuggerPart1
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib/idea_rt.jar=53445 -Dfile.encoding=U
average:180
Process finished with exit code 0
DebuggerExample > src > DebuggerPart1
23:1 LF UTF-8 4 spaces


DebuggerExample
Version control
Current File
Project
DebuggerExample ~/IdeaP
.idea
out
src
DebuggerPart1
.gitignore
DebuggerExample.iml
External Libraries
Scratches and Consoles
DebuggerPart1.java
public class DebuggerPart1 {
public static void main(String[] args){ args: []
int zahl1 = 10; zahl1: 10
int zahl2 = 20; zahl2: 20
int zahl3 = 30; zahl3: 30
int summe = zahl1+zahl2+zahl3; summe: 60 zahl2: 20 zahl3: 30 zahl1: 10
int anzahl = 3; anzahl: 3
int average = findAverage(summe, anzahl); summe: 60 anzahl: 3
System.out.println("average:"+average);
}
public static int findAverage(int summe, int anzahl){ 1 usage
int result = summe * anzahl; //Es sollte summe/anzahl sein
return result;
}
}
Debug
DebuggerPart1
Threads & Variables
Console
"main"@1 i...n": RUNNING
Evaluate expression (⏎) or add a watch (⇧⌘⏎)
main:11, DebuggerPart1
args = {String[0]@823} []
zahl1 = 10
zahl2 = 20
zahl3 = 30
summe = 60
anzahl = 3
Switch frames from anywhere in the IDE wi...
DebuggerExample > src > DebuggerPart1
11:1 LF UTF-8 4 spaces

- **Blaue Zeile:** alles **bis und ohne** diese Zeile wurde ausgeführt
- **Debugger Symbole:**



View breakpoints
- Alle breakpoints anzeigen

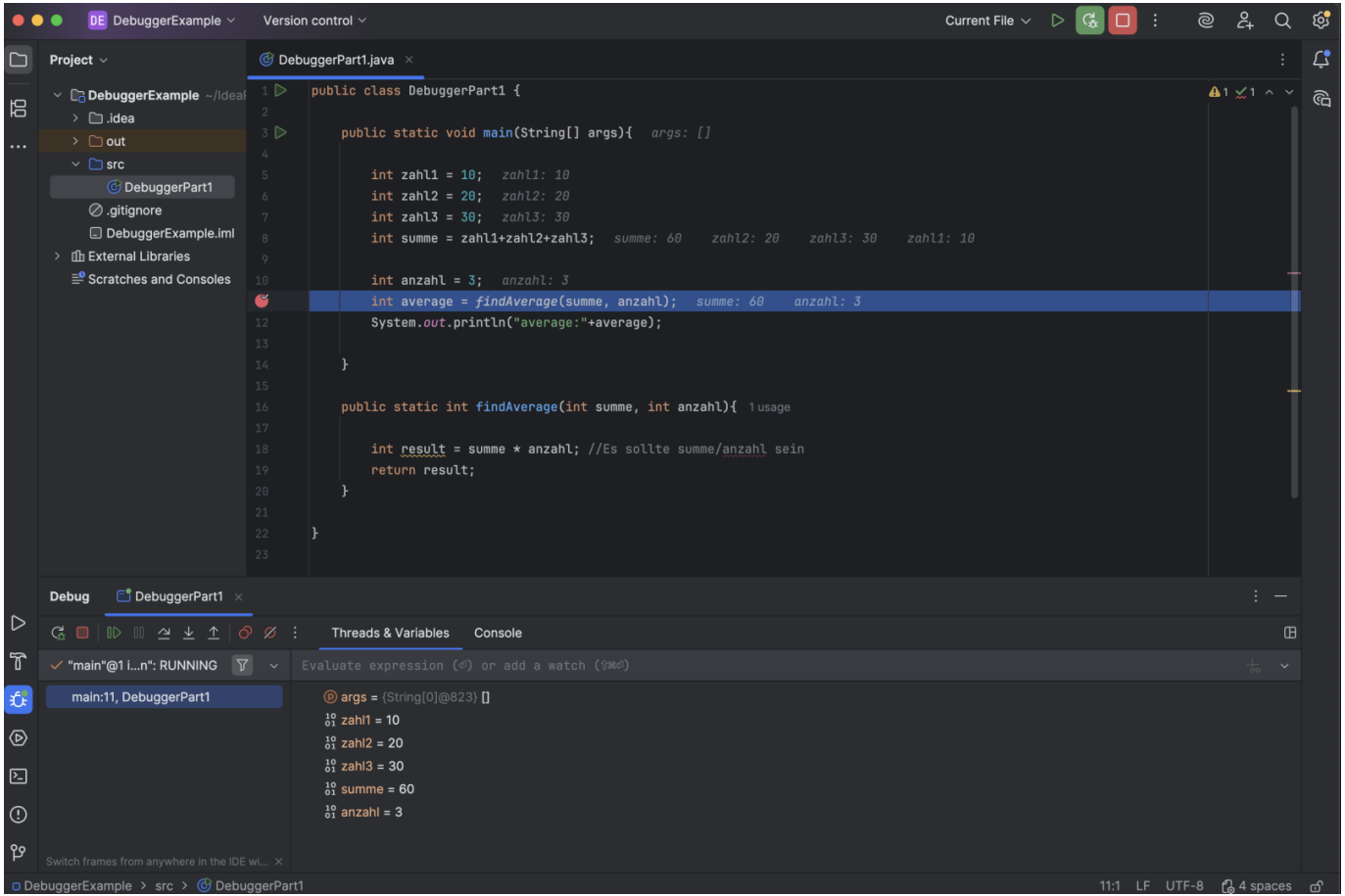
Step out
- Um aus einer Methode raus zu treten

Termination
- Um den Debugger zu stoppen

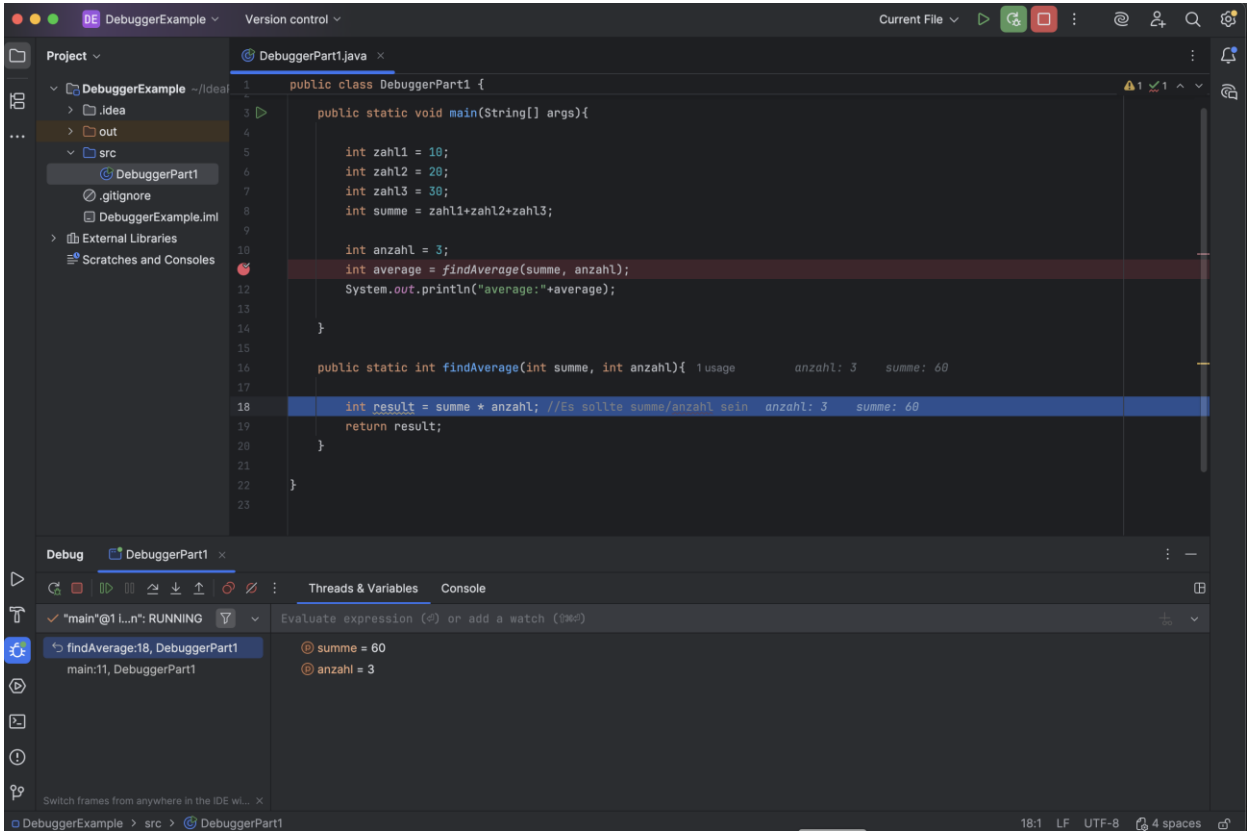
Step over
- Um zur nächsten Zeile zu gehen.

Step into
- Um in eine Methode zu springen.

Von dieser Zeile aus weitergehen…

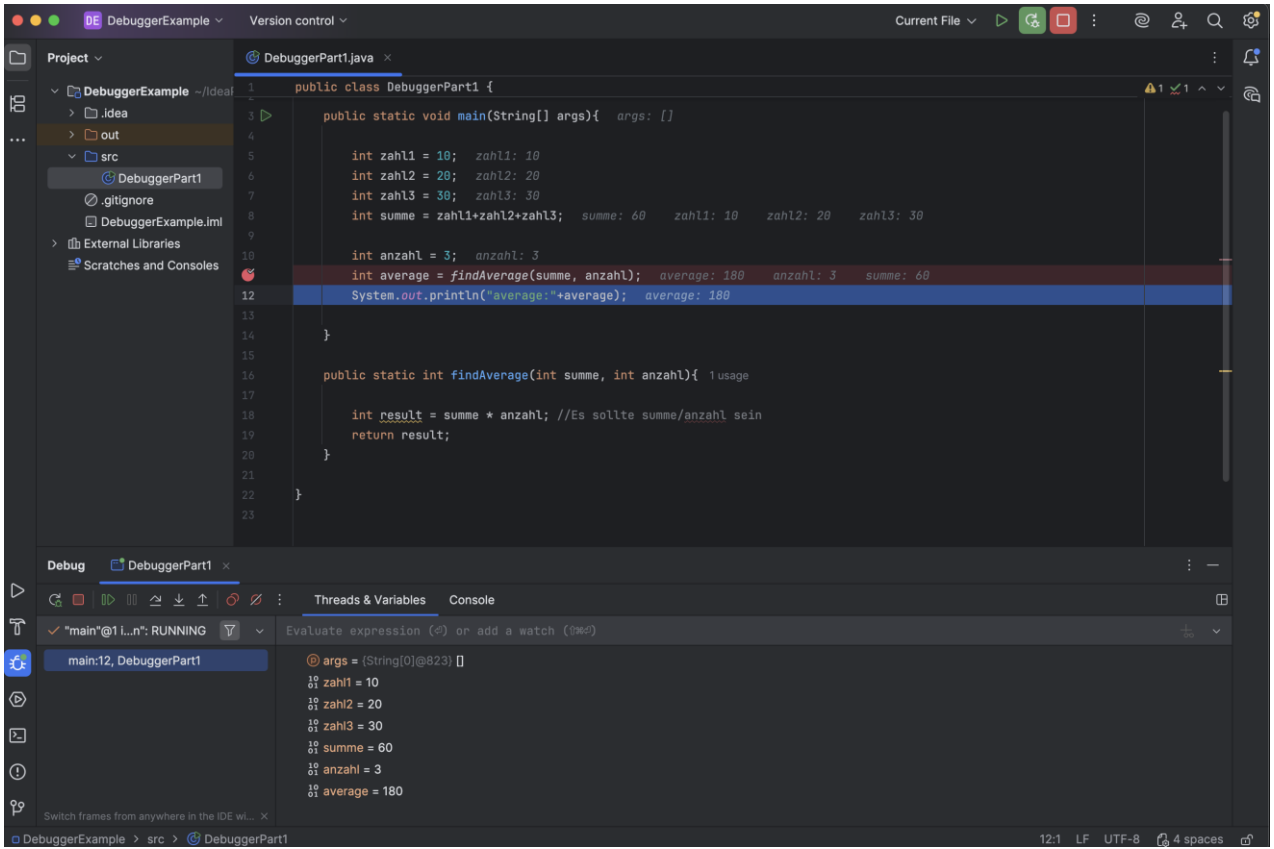


Mit step into:



Wir sind in die aufgerufene Methode **findAverage** hineingegangen und können nun diese Methode Schritt für Schritt ausführen.

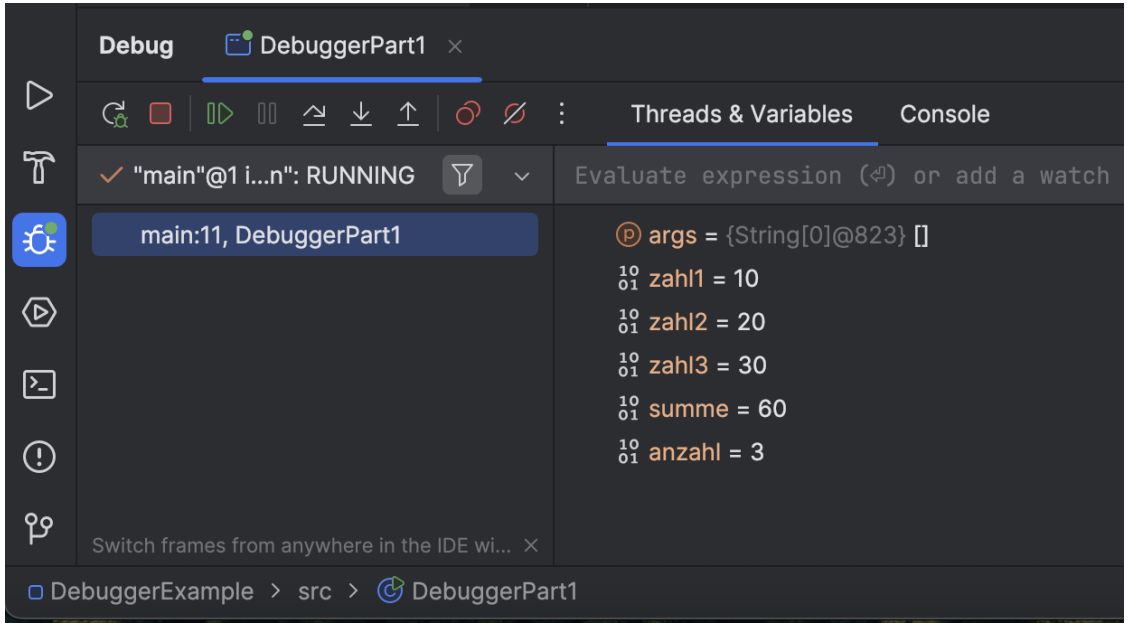
Mit step over:



- Der Methodenaufruf von **findAverage** wurde in einem Schritt ausgeführt und der Rückgabewert an **average** zugewiesen.
- Wir befinden uns jetzt in der nächsten Zeile.

Step over vs. Step into: Wann welches benutzen?

- **Step over:** 
  - Zur nächsten Zeile springen.
  - Wenn in der Zeile eine Methode aufgerufen wird, wird sie ausgeführt. Man geht davon aus, dass sie wie erwartet funktioniert und dass der Rückgabewert korrekt ist.
- Step into: 
  - In die Methode hineingehen, um sie schrittweise zu durchlaufen.
  - Dies verwendet man, wenn man unsicher ist, ob die Methode wie erwartet funktioniert, und man den Ablauf genauer überprüfen möchte.



- Zeigt alle Variablen und ihre Werte an, die im Scope der blauen Zeile liegen.
- Nach der Ausführung jeder Zeile kann man überprüfen, ob sich die Werte wie erwartet geändert haben.

```java
public class DebuggerPart1 {

    public static void main(String[] args){   args: []

        int zahl1 = 10;   zahl1: 10
        int zahl2 = 20;   zahl2: 20
        int zahl3 = 30;   zahl3: 30
        int summe = zahl1+zahl2+zahl3;   summe: 60    zahl2: 20    zahl3: 30    zahl1: 10

        int anzahl = 3;   anzahl: 3
        int average = findAverage(summe, anzahl);   anzahl: 3    summe: 60
        System.out.println("average:"+average);

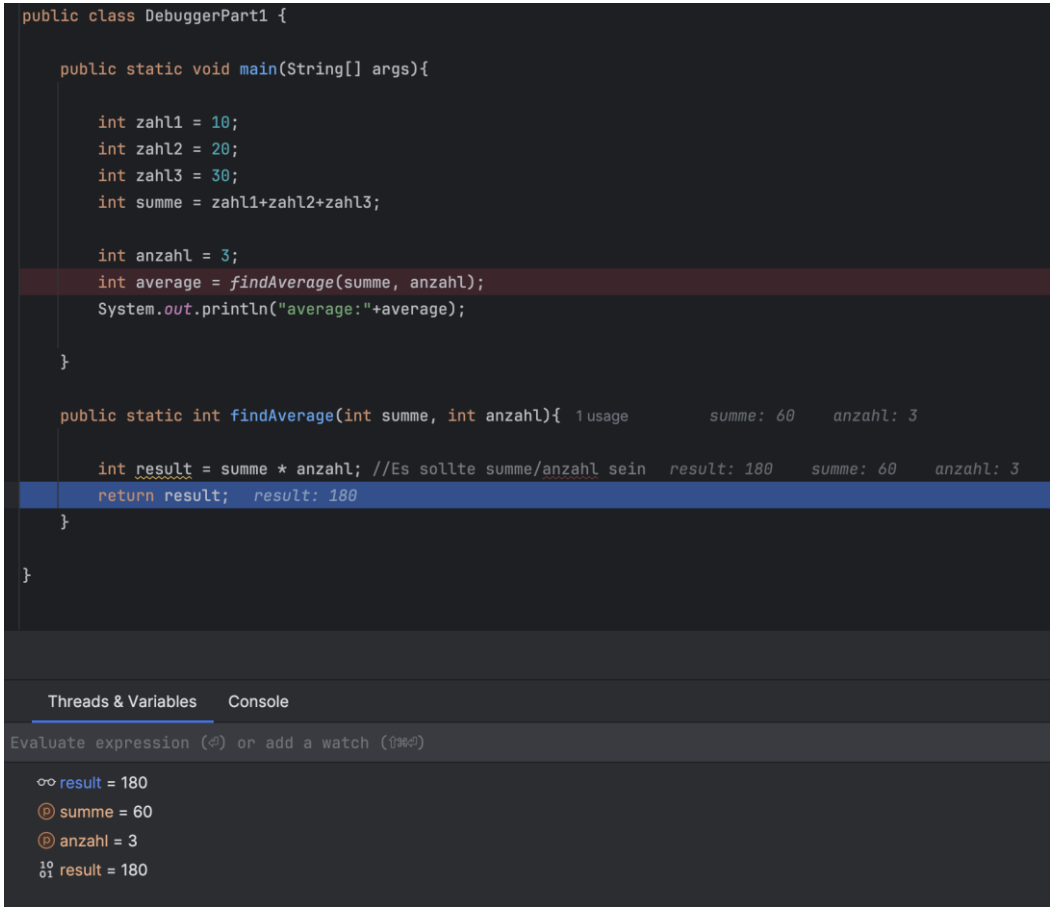
    }

    public static int findAverage(int summe, int anzahl){  1 usage

        int result = summe * anzahl; //Es sollte summe/anzahl sein
        return result;
    }

}
```

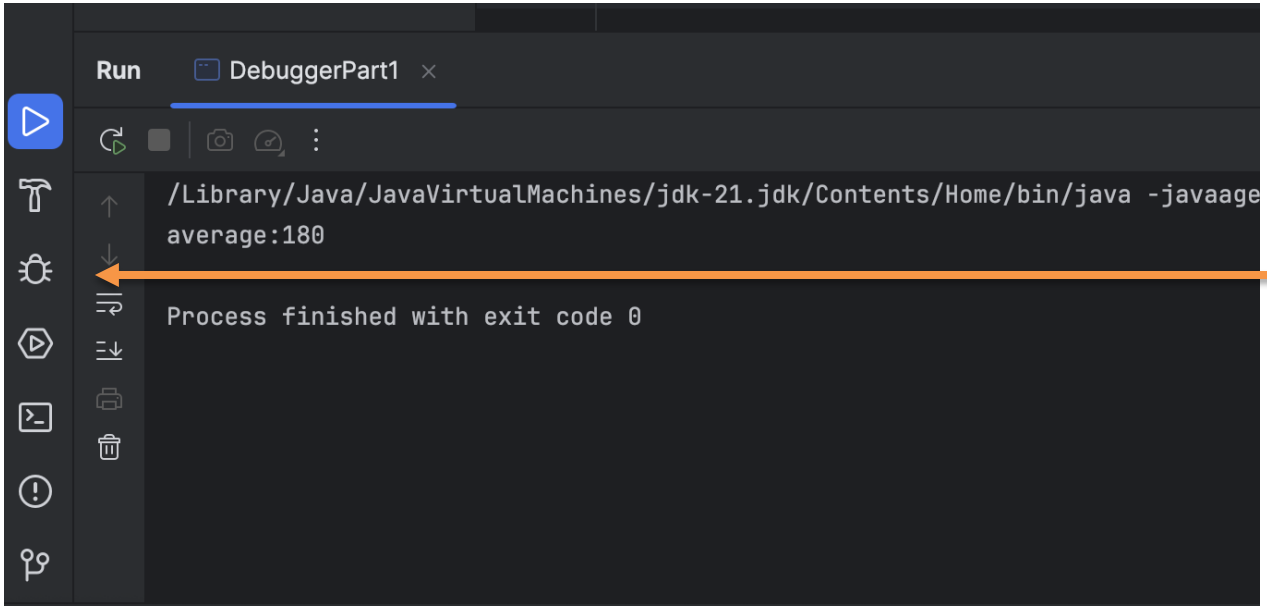
- Die Werte der Variablen werden auch am Ende jeder Zeile in Grau angezeigt



Hier kann man sehen, welche Werte **summe**, **anzahl** und **average** enthalten.

Unter „Threads & Variables“ erkennt man, dass **average** durch **(summe * anzahl)** berechnet wird, anstatt durch **(summe / anzahl)**.

Falls nicht automatisch Threads & Variables gezeigt wird:



Debug auswählen

Kahoot

https://create.kahoot.it/share/u5-eprog-25/f1ab6755-e8d9-44b5-a98d-585d07895edb

Vorbesprechung

Aufgabe 1: Sieb des Eratosthenes



Schreiben Sie ein Programm "Sieb.java", das eine Zahl *limit* einliest und die Anzahl der Primzahlen, die grösser als 1 und kleiner oder gleich dem *limit* sind, ausgibt. Dazu ermitteln Sie in einem ersten Schritt alle Primzahlen, die kleiner oder gleich *limit* sind. Dieses Teilproblem können Sie mit dem Sieb des Eratosthenes lösen. Das Sieb des Eratosthenes findet Primzahlen bis n. Man betrachtet alle Zahlen von 2 bis `n` und streicht zuerst alle Vielfachen der ersten Zahl (2). Dann geht man zur nächsten ungestrichenen Zahl (3) und wiederholt das Streichen ihrer Vielfachen. Das macht man, bis man dies für alle Zahlen gemacht hat. Sie können ein `Boolean`-Array verwenden, um zu speichern, welche Zahlen Primzahlen sind und welche nicht. Übrig bleiben die Primzahlen. Danach können Sie die Anzahl der gefundenen Primzahlen anhand dieses Arrays bestimmen.

Beispiel: Für $limit = 13$ sollte Ihr Programm 6 ausgeben (Primzahlen: 2, 3, 5, 7, 11, 13).

Hinweis: Es ist nicht zwingend nötig von 2 bis n zu gehen. Von 2 bis $\sqrt{n}$ zu gehen reicht bereits aus, da eine Zahl $\leq n$ nicht einen Teiler grösser als $\sqrt{n}$ ausser sich selbst haben kann.

Aufgabe 2: Arrays



1. Implementieren Sie die Methode `ArrayUtil.zeroInsert(int[] x)` in der Datei "ArrayUtil.java". Die Methode nimmt einen Array `x` als Argument und gibt einen Array zurück. Der zurückgegebene Array soll die gleichen Werte wie `x` haben, ausser: Wenn eine positive Zahl direkt auf eine negative Zahl folgt oder wenn eine negative Zahl direkt auf eine positive Zahl folgt, dann wird dazwischen eine 0 eingefügt.

   Beispiele:

   - Wenn `x` gleich `[3, 4, 5]` ist, dann wird `[3, 4, 5]` zurückgegeben.
   - Wenn `x` gleich `[3, 0, -5]` ist, dann wird `[3, 0, -5]` zurückgegeben.
   - Wenn `x` gleich `[-3, 4, 6, 9, -8]` ist, dann wird `[-3, 0, 4, 6, 9, 0, -8]` zurückgegeben.

   Versuchen Sie, die Methode rekursiv zu implementieren.

Aufgabe 2: Arrays



2. Implementieren Sie die Methode `ArrayUtil.tenFollows(int[] x, int index)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn im Array `x` ab Index `index` der zehnfache Wert einer Zahl `n` direkt der Zahl `n` folgt. Dies muss nur für das erste Auftreten der Zahl `n` ab Index `index` im Array `x` geprüft werden. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele:

   - `tenFollows([1, 2, 20], 0)` gibt `true` zurück.
   - `tenFollows([1, 2, 7, 20], 0)` gibt `false` zurück.
   - `tenFollows([3, 30], 0)` gibt `true` zurück.
   - `tenFollows([3], 0)` gibt `false` zurück.
   - `tenFollows([1, 2, 20, 5], 1)` gibt `true` zurück.
   - `tenFollows([1, 2, 20, 5], 2)` gibt `false` zurück.

   Die `main` Methode in `ArrayUtil` gibt die oben genannten Beispielaufrufe sowie das entsprechende Ergebnis der jeweiligen Methode aus. Hiermit können Sie überprüfen, ob Ihre Implementierungen die richtigen Ergebnisse zurückliefern. In "ArrayUtilTest.java" im Ordner "test" in der Übungsvorlage finden Sie zusätzlich einige Unit-Tests für beide Methoden (für eine detaillierte Beschreibung zu automatisiertem Testen und der Ausführung solcher Tests siehe Aufgabe 3). Sie können die `main` Methode und die Tests beliebig abändern und/oder mit Ihren eigenen Inputs erweitern.

Aufgabe 3: 2D Arrays



Gegeben einer Matrix M, prüfen Sie zuerst ob diese eine $n \times n$ Matrix ist, deren Elemente positive ganze Zahlen sind. Danach prüfen Sie ob zusätzlich alle Zahlen kleiner gleich n^2 sind. Somit gilt nun $0 < m_{i,j} \leq n^2$. Prüfen Sie ebenfalls, ob die Elemente der Matrix jeweils genau einmal vorkommen, sprich ob $m_{x,y} = m_{p,q} \Rightarrow (x = p) \wedge (y = q)$ gilt. Wir sagen, dass die Matrix M *perfekt* ist, wenn zusätzlich alle Zeilensummen und Spaltensummen gleich sind (also $\sum_{k=0}^{k=n-1} m_{i,k} = \sum_{k=0}^{k=n-1} m_{j,k}$ für alle i, j und $\sum_{k=0}^{k=n-1} m_{k,i} = \sum_{k=0}^{k=n-1} m_{k,j}$ für alle i, j mit $0 \leq i, j < n$).

Vervollständigen Sie die Methode `boolean checkMatrix(int[][] m)` von der Klasse Matrix, so dass diese Methode `true` zurückgibt wenn die Input Matrix *perfekt* ist, und `false` sonst. Sie können davon ausgehen, dass der Parameter `m` nicht `null` ist. Alle anderen Eigenschaften müssen Sie selber testen. Eine Matrix ist nur perfekt, wenn alle genannten Eigentschaften gelten.

Testen Sie Ihr Programm ausgiebig - am besten mit JUnit - und pushen Sie die Lösung vor dem Abgabetermin. Wir haben Ihnen einen JUnit Test in der Klases `MatrixTest` bereits erstellt.

Aufgabe 4: Testen mit JUnit



Zweck des Programms:

- Wochentag eines Datums (nach 01.01.1900) ausgeben
  Beispiel:13.10.2017 → Friday
  Gibt fälschlicherweise aber *“The 13.10.2017 is a Sunday”* aus.
- Berücksichtigt Schaltjahre (“Leap year”)

Funktionsweise:

1. Überprüft, ob Datum OK ist
2. Zählt die Tage ab 1.1.1900 bis zum eingegebenen Datum
3. Wochentag = Tage % 7

Aufgabe 4: Testen mit JUnit

Tests in *PerpetualCalendarTest.java*

- Einzelne Tests prüfen Rückgabewerte von einzelnen Methoden des Programms *PerpetualCalendar.java*
- Tests sollten *interessante* Parameter für die Methoden testen
- Beispiel `testCountDaysInYear()`:
  `assertEquals(366, PerpetualCalendar.countDaysInYear(1904));`
  1904 ist ein Schaltjahr, also sollte `countDaysInYear()` 366 Tage zurückgeben

Aufgabe 5: Matching Numbers



Implementieren Sie die Methode `Match.matchNumber(long A, int M)`. Die Methode soll für eine Zahl `A` und eine nicht-negative drei-stellige Zahl `M` die Position von `M` in `A` zurückgeben. Sei `M` eine Zahl mit den Ziffern $M_2M_1M_0$ (das heisst, es gilt $M = M_0 + 10 \cdot M_1 + 100 \cdot M_2$), wobei jede Ziffer 0 sein kann. Zusätzlich sei `A` eine Zahl, sodass A_i die i-te Ziffer von `A` ist (das heisst, es gilt $|A| = \sum_i 10^i \cdot A_i$), wobei `A` unendlich viele führende Nullen hat. Die Position von `M` in `A` ist die kleinste Zahl j, sodass $A_j = M_0$ und $A_{j+1} = M_1$ und $A_{j+2} = M_2$ gilt. Die Methode soll `-1` zurückgeben, falls es kein solches j gibt.

Beispiele:

`matchNumber(32857890, 789)` soll `1` zurückgeben.
`matchNumber(37897890, 789)` soll `1` zurückgeben.
`matchNumber(1800765, 7)` soll `2` zurückgeben.
`matchNumber(1800765, 8)` soll `-1` zurückgeben (die drei Ziffern von `8` sind `008`).
`matchNumber(75, 7)` soll `1` zurückgeben (da `007` and Position `1` von `0075` ist).

Aufgabe 5: Matching Numbers

Implementieren Sie die Berechnung in der Methode `int matchNumber(long A, int M)`, welche sich in der Klasse `Match` befindet. Die Deklaration der Methode ist bereits vorgegeben. Sie können davon ausgehen, dass $0 \leq \texttt{M} < 1000$ gilt.

In der `main` Methode der Klasse `Match` finden Sie die oberen Beispiele als kleine Tests, welche Beispiel-Aufrufe zur `matchNumber`-Methode machen und welche Sie als Grundlage für weitere Tests verwenden können. In der Datei `MatchTest.java` geben wir die gleichen Tests zusätzlich auch als JUnit Test zur Verfügung. Sie können diese ebenfalls nach belieben ändern. Es wird *nicht* erwartet, dass Sie für diese Aufgabe den JUnit Test verwenden.

Tipp: Die Methode `Integer.toString(int i)` wandelt einen Integer in einen String um.

Aufgabe 6: Substring-Counter (Bonus!)



- Diese Aufgabe gibt Bonuspunkte!
- Regeln findet ihr hier: https://lec.inf.ethz.ch/infk/eprog/2026/exercises/additionals/bonusaufgaben_regeln.pdf